INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES
DE MONTERREY
CAMPUS GUADALAJARA

Curso de ROS 2
Apuntes de clase

Impartido por:

Dr. Eduardo de Jesús Dávila Meza

para el bloque integrador de:

Robótica y Sistemas Inteligentes

para estudiantes de:

Ingeniería en Robótica y Sistemas Digitales

de la Escuela de:

Ingeniería y Ciencias

Zapopan, Jalisco, México. Marzo - Junio 2026.

Contents

List of Acronyms and Special Terms

Acronyms

S

SLAM simultaneous localization and mapping. — Pages: 173, 174, 181, 182, 186.

CHAPTER 1

ROS 2 Environment Setup and Development Tools

Dr. Eduardo de Jesús Dávila Meza
EduardoDavilaMeza
eduardo.davila.meza@tec.mx

1.1 Introduction to ROS 2

1.1.1 Basic Concepts

The Robot Operating System (ROS) is not an actual operating system but a flexible *framework* for writing robot software. It provides a collection of software libraries, tools, and conventions to simplify the task of creating complex and robust robot applications. From drivers and state-of-the-art algorithms to powerful developer tools, ROS has the open source tools you need for your next robotics project. Some of its main advantages include:

- **Multi-language:** While ROS supports multiple programming languages, the primary supported languages are C++ and Python. Support for other languages like Java exists but is less common.
- **Free and open-source:** ROS 1 and ROS 2 are available on Ubuntu, with ROS 2 also supporting Windows and macOS. However, the availability and stability can vary across different versions and platforms.
- **Support:** Since ROS was started in 2007, a lot has changed in the robotics and ROS community. The ROS 2 project began in 2015 to address the evolving needs of the robotics community, building upon the strengths of ROS 1 and introducing improvements for better performance, security, and support for real-time systems.

1.1.2 Some ROS Distributions

ROS is released as distributions (or "distros"), with several versions supported concurrently. Some releases come with long-term support (LTS), meaning they are more stable and have undergone extensive testing, while other distributions are newer, have shorter lifetimes, and support more recent platforms and package versions. Generally, a new ROS distro is released every year on World Turtle Day, with LTS releases appearing in even-numbered years. ROS is available for different versions of Ubuntu and other operating systems. Some of the notable distributions include:

- **Indigo Igloo (ROS 1):** Ubuntu 14.04 (Trusty Tahr)
- **Kinetic Kame (ROS 1):** Ubuntu 16.04 (Xenial Xerus)
- **Melodic Morenia (ROS 1):** Ubuntu 18.04 (Bionic Beaver)
- **Noetic Ninjemys (ROS 1):** Ubuntu 20.04 (Focal Fossa)
- **Foxy Fitzroy (ROS 2):** Ubuntu 20.04 (Focal Fossa)
- **Humble Hawksbill (ROS 2):** Ubuntu 22.04 (Jammy Jellyfish)
- **Jazzy Jalisco (ROS 2):** Ubuntu 24.04 (Noble Numbat)

For more details, see the ROS Distributions list. Currently, the Noetic Ninjemys, Humble Hawksbill, and Jazzy Jalisco distributions are actively supported.

1.1.3 ROS Distribution for the Course

For this course, we will use **ROS 2 Humble Hawksbill**, a long-term support (LTS) release that offers enhanced performance, security, and real-time capabilities. Its logo, shown in Figure 1.1, reflects its distinctive branding. ROS 2 Humble Hawksbill is compatible with Ubuntu 22.04 (Jammy Jellyfish) and Windows 10, making it a versatile choice for both simulation and deployment on physical hardware. This distribution will serve as the foundation for all class exercises and, in particular, for project development.

Figure 1.1: ROS 2 Humble Hawksbill Logo.

1.2 Installation of Ubuntu 22 and ROS 2 Humble Hawksbill

1.2.1 Installing Ubuntu 22.04.5 LTS (Jammy Jellyfish)

1. **Download the Universal USB Installer**
   Visit the Uptodown website to download the Universal USB Installer, version `2.0.1.4`, on Windows.
2. **Download the Ubuntu ISO**
   Download the Ubuntu 22.04.5 LTS (Jammy Jellyfish) ISO - *64-bit PC (AMD64) desktop image*, from the official release page: Ubuntu releases.
3. **Create a Bootable USB Drive**
   Insert a USB drive (minimum 8 GB) and use the Universal USB Installer tool with the downloaded ISO to create a bootable USB stick. Follow the on-screen prompts to properly set up the drive. You may also follow the instructions provided by your instructor, the official guide: Universal USB Installer, or refer to this tutorial: YouTube Guide.
4. **Prepare Your Machine (if dual-booting with Windows)**
   If you are keeping Windows and installing Ubuntu alongside it, you should free some unallocated disk space before installation:
   - Defragment the `C:` drive.
   - Additionally, you can shrink the Windows partition to create at least 32 GB of unallocated space (advanced installation).
   - If the volume reduction fails, delete old restore points to free up additional space. See, for example, this tutorial: Windows - Shrinking hard disk volume to create new partition.
5. **Install Ubuntu on Your Machine**
   Boot the target computer from the USB drive. Follow the Ubuntu installation wizard to install Ubuntu 22.04 LTS and configure your system settings as needed:
   - Select language and keyboard layout.
   - Connect to a network (optional during installation).
   - Choose installation type (*Normal* or *Minimal*).
   - Allocate disk space (at least 64 GB) if you want to *Install Ubuntu alongside Windows* or select *Erase disk and install Ubuntu*.
   - Create a user account, password, and set the hostname.

1.2.2 First Steps After Installing Ubuntu 22

Open a terminal from the application menu by typing `terminal` or by pressing `Ctrl+Alt+T`, and then:

- Update the system:

```
$ sudo apt update && sudo apt upgrade -y
```

- Check the installed Python version:

```
$ python3 --version
```

1.2.3 Installing ROS 2 Humble Hawksbill

Recommended Method: Using Debian Packages

The official ROS 2 Humble Hawksbill documentation recommends installing from deb packages for a stable and straightforward setup. Follow the installation instructions on the official ROS 2 Humble Hawksbill Installation Guide. This method installs pre-built binaries and generally covers all core dependencies needed for running ROS 2.

Instructions:

Set locale
Make sure you have a locale which supports UTF-8. The following settings are tested, although any UTF-8 supported locale should work.

```
$ locale  # check for UTF-8

$ sudo apt update && sudo apt install locales
$ sudo locale-gen en_US en_US.UTF-8
$ sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
$ export LANG=en_US.UTF-8

$ locale  # verify settings
```

Setup Sources
You will need to add the ROS 2 apt repository to your system. First, ensure that the Ubuntu Universe repository is enabled.

```
$ sudo apt install software-properties-common
$ sudo add-apt-repository universe
```

Now, add the ROS 2 GPG key with apt:

```
$ sudo apt update && sudo apt install curl -y
$ sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
↪  /usr/share/keyrings/ros-archive-keyring.gpg
```

Then, add the repository to your sources list:

```
$ echo "deb [arch=$(dpkg --print-architecture)
↪  signed-by=/usr/share/keyrings/ros-archive-keyring.gpg]
↪  http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME)
↪  main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

Install ROS 2 Packages
Update your apt repository caches after setting up the repositories:

```
$ sudo apt update
```

ROS 2 packages are built on frequently updated Ubuntu systems. It is always recommended to ensure your system is up to date before installing new packages:

```
$ sudo apt upgrade
```

⚠ Warning:

Due to early updates in Ubuntu 22.04, it is important that `systemd` and `udev`-related packages are updated before installing ROS 2. Installing ROS 2's dependencies on a freshly installed system without upgrading can trigger the removal of critical system packages. Please refer to ros2/ros2#1272 and Launchpad #1974196 for more information.

Desktop Install (Recommended): This includes ROS, RViz, demos, and tutorials.

```
$ sudo apt install ros-humble-desktop
```

Development Tools: Compilers and other tools to build ROS packages.

```
$ sudo apt install ros-dev-tools
```

Environment Setup – Sourcing the Setup Script

Set up your environment by sourcing the following file (replace `.bash` with your shell if not using bash):

```
$ source /opt/ros/humble/setup.bash
```

Alternative: Building from Source

If you require customizations or intend to develop complex packages, you can also install ROS 2 Humble Hawksbill from source. Note that building from source may require additional dependency installations (similar to ROS1). For most beginners and for the purposes of this course, the deb package installation is recommended.

Sourcing the Setup Script

After installing ROS 2, source the setup script to ensure your environment is correctly configured:

```
$ source /opt/ros/humble/setup.bash
```

Optionally, add the sourcing command to your shell's initialization file (e.g., `.bashrc`) so that the ROS environment variables are automatically loaded every time a new terminal session is opened (replace `.bash` with your shell if not using bash).

```
$ echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
$ source ~/.bashrc
```

1.3 Try Some Examples

1.3.1 Talker-Listener

If you installed `ros-humble-desktop`, try some examples. In one terminal, source the setup file and run a C++ talker:

```
$ source /opt/ros/humble/setup.bash
$ ros2 run demo_nodes_cpp talker
```

In another terminal, source the setup file and run a Python listener:

```
$ source /opt/ros/humble/setup.bash
$ ros2 run demo_nodes_py listener
```

You should see the talker publishing messages and the listener confirming reception, verifying that both the C++ and Python APIs are working properly.

1.3.2 Turtlesim: Publish and Move a Turtle

Another example you can try is the `turtlesim` package, which demonstrates basic ROS 2 communication using a simulated turtle.

First, ensure that the package is installed:

```
$ sudo apt install ros-humble-turtlesim
```

Then, in a new terminal, launch the turtlesim node:

```
$ ros2 run turtlesim turtlesim_node
```

Next, open another terminal, use the `teleop` node to control the turtle with your keyboard:

```
$ ros2 run turtlesim turtle_teleop_key
```

Now, pressing the arrow keys will move the turtle in the corresponding direction. This example demonstrates publishing velocity commands to a topic that the turtlesim node subscribes to, allowing interaction with the simulated environment.

1.4 `colcon` Configuration

`colcon` is the command-line tool used in ROS 2 to build sets of packages in a workspace. It automatically detects packages (via `package.xml`), resolves dependencies, and builds them (often in parallel) to simplify the development process. `colcon` also supports options like `--symlink-install` for faster iterative development.

1.4.1 Installing `colcon`

Update your package list and ensure that the common `colcon` extensions are installed. Although these packages are usually installed as part of the `ros-humble-desktop` and `ros-dev-tools` installations, it is good practice to verify their presence by running the following commands:

```
$ sudo apt update
$ sudo apt install python3-colcon-common-extensions
```

1.4.2 Enabling `colcon` Argument Completion

To simplify command usage with `colcon`, add the argument completion script to your shell initialization file (e.g., `.bashrc`). Execute the following commands:

```
$ echo "source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash" >> ~/.bashrc
$ source ~/.bashrc
```

You can verify that the sourcing command was added by viewing your `.bashrc` file:

```
$ cat ~/.bashrc
```

1.5 Configuring the Development Environment

With the following tools and extensions, your development environment will be well-equipped for Python projects.

1.5.1 Installing Visual Studio Code

Visual Studio Code (VS Code) is a versatile code editor that supports various programming languages and development environments. To install it on Ubuntu 22.04, follow these steps:

1. **Using the Ubuntu Software Center**
   (a) Open the *Ubuntu Software* application from the application menu.
   (b) In the search bar, type `Visual Studio Code`.
   (c) Locate Visual Studio Code in the search results and click *Install*.
2. **Using the Official .deb Package**
   (a) Visit the official VS Code download page: code.visualstudio.com.
   (b) Click on the `.deb` package suitable for Debian/Ubuntu.
   (c) Once downloaded, open a terminal (`Ctrl+Alt+T`) and navigate to the directory containing the downloaded file.
   (d) Install the package using:

   ```
   $ sudo apt install ./<file>.deb
   ```

   Replace `<file>` with the actual filename.

These methods ensure that VS Code is added to your system repositories and receives updates automatically.

1.5.2 Installing Recommended VS Code Extensions

Enhance your development experience by installing the following VS Code extensions:

- **C++** by *Microsoft* – Offers C++ IntelliSense, debugging, and code browsing.
- **CMake** by *twxs* – Simplifies working with CMake projects.
- **CMake Tools** by *Microsoft* – Provides CMake project integration.
- **Error Lens** by *Alexander* – Displays inline error messages in the code editor.
- **Indent-Rainbow** by *oderwat* – Highlights indentation levels with different colors.
- **Python** by *Microsoft* – Provides rich support for Python, including features such as IntelliSense, linting, and debugging.
- **Robotics Developer Environment** by *Ranch Hand Robotics LLC* – Helps build robotics solutions targeting the ROS 2.
- **URDF** by *smilerobotics* – Adds syntax highlighting and support for URDF files.

- **URDF Visualizer** by *morningfrog* – Allows to preview URDF files within the editor.
- **vscode-icons** by *VSCode Icons Team* – Adds file icons for better visual identification.
- **XML** by *Red Hat* – Enhances XML editing capabilities.
- **XML Tools** by *Josh Johnson* – Offers additional functionalities for XML files.

To install these extensions:

1. Open VS Code from the application menu by typing `code`.
2. Navigate to the *Extensions* view by clicking on the square icon () in the sidebar or pressing `Ctrl+Shift+X`.
3. Search for each extension by name and click *Install*.

1.5.3 Installing Terminator for Multiple Terminals

Terminator is a terminal emulator that allows splitting the window into multiple terminals, facilitating simultaneous operations. To install Terminator:

1. Open a terminal.
2. Update the package list:
   ```
   $ sudo apt update
   ```
3. Install Terminator:
   ```
   $ sudo apt install terminator
   ```
4. Launch Terminator from the applications menu, by typing `terminator` in the terminal, or by pressing `Ctrl+Alt+T`.

Installing Terminator may have set it as the default, so when you press `Ctrl+Alt+T` it opens Terminator instead of GNOME Terminal (or your original terminal). To achieve your goal, you can change the default and then assign Terminator to a different shortcut, as described in the next.

Reset the Default Terminal Emulator

Ubuntu uses the alternative system for the "x-terminal-emulator." You can reconfigure it so that pressing `Ctrl+Alt+T` launches your original terminal (for example, GNOME Terminal):

1. **Open a terminal** (if Terminator is the only one available, you can temporarily launch it).
2. Run the command:
   ```
   $ sudo update-alternatives --config x-terminal-emulator
   ```
3. You will see a list of installed terminal emulators. Type the number corresponding to your original terminal (e.g., `/usr/bin/gnome-terminal.wrapper`) and press **Enter**.

This sets the default terminal emulator that the system shortcut uses.

Create a Custom Shortcut for Terminator

Next, if you want a separate shortcut (like `Ctrl+Alt+E`) that launches Terminator:

1. **Open Settings** in your Ubuntu desktop.
2. Navigate to **Keyboard** (or **Keyboard Shortcuts**).
3. Scroll down to or click on **Custom Shortcuts**.
4. Click the **Add Shortcut** button.
5. Set a name (e.g., "Terminator") and for the command enter:
   ```
   terminator
   ```
6. Assign the new shortcut to `Ctrl+Alt+E` (click the shortcut key field and press the keys).
7. Save your new shortcut.

Now:

- `Ctrl+Alt+T` will launch your default terminal (GNOME Terminal),
- `Ctrl+Alt+E` will launch Terminator.

This way, you keep your usual terminal for everyday tasks and have Terminator available on a separate key combination for when you need its advanced features.

1.6 Practice Assignment: Running ROS 2 Examples

In this assignment, you will run the ROS 2 examples presented in Section 1.3 on your computer and capture evidence of successful execution using a screenshot. Submit your evidence in the designated assignment area on Canvas (within the ROS 2 module) as a PNG file. The filename must follow this format:

`FirstnameLastname_evidence_sX.png`,

where `sX` indicates the session number, for example:

`EduardoDavila_evidence_s1.png`.

1.6.1 Examples to Try

Talker-Listener

After installing `ros-humble-desktop` (and optionally `Terminator`), try the following:

- In **Terminal 1**, source the setup file and run a C++ talker:

```
$ source /opt/ros/humble/setup.bash
$ ros2 run demo_nodes_cpp talker
```

- In **Terminal 2**, source the setup file and run a Python listener:

```
$ source /opt/ros/humble/setup.bash
$ ros2 run demo_nodes_py listener
```

You should observe that the talker publishes messages and the listener confirms their reception.

Turtlesim: Publish and Move a Turtle

Another example is provided by the `turtlesim` package:

- In **Terminal 3**, run the turtlesim node:

```
$ ros2 run turtlesim turtlesim_node
```

- In **Terminal 4**, run the teleoperation node to control the turtle:

```
$ ros2 run turtlesim turtle_teleop_key
```

Use the arrow keys to move the turtle. This example demonstrates publishing velocity commands to control the simulated turtle.

1.6.2 Submission Instructions

1. Run the examples as described above.
2. Capture a **screenshot showing the terminal output** where the examples are running successfully (see Figure 1.2 for an example).
3. Save the screenshot in PNG format.

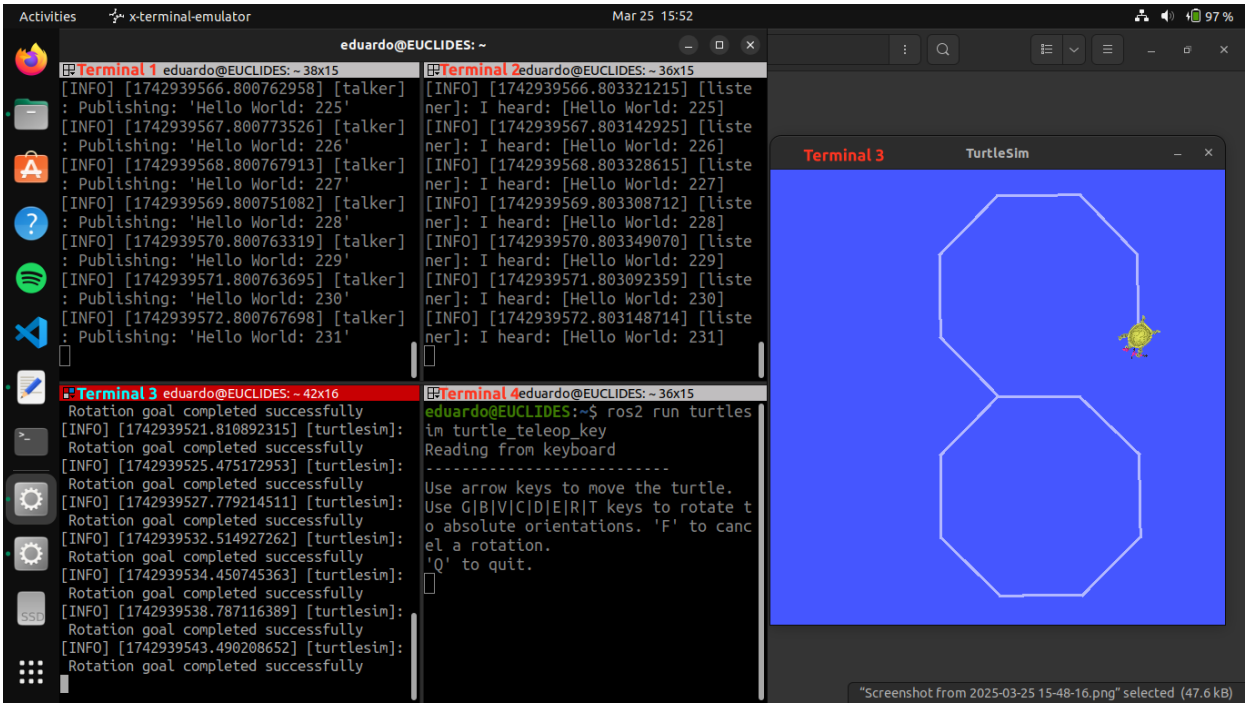

Figure 1.2: Example screenshot showing terminal outputs from both the Talker-Listener example and the Turtlesim node example in action.

4. Name the file according to the format: `FirstnameLastname_evidence_sX.png` (replace `X` with the session number).
5. Submit your screenshot file as instructed by the course guidelines.

Note: Your machine's username and device name must be visible in the screenshot to verify the authenticity of the submission, as shown in Figure 1.2. Evidence that appears copied, unclear, or altered will be considered invalid and may result in a score of 0 for this assignment.

APPENDIX A

Essential ROS 2 Command Reference

Author: Dr. Eduardo de Jesús Dávila Meza.
EduardoDavila-AI-PhD

This appendix compiles the commands utilized throughout the course for creating and configuring workspaces, packages, nodes, and using integrated tools in ROS 2.

A.1 System Package Management and Updates in Ubuntu

Command	Description
`sudo apt update`	Updates the list of available packages and their versions.
`sudo apt upgrade`	Installs the latest versions of all installed packages.
`sudo apt install <pkg_name>`	Installs the specified package.
`sudo apt install ./<file>.deb`	Installs a package from a local `.deb` file.
`sudo apt install ros-humble-desktop`	Installs the full desktop version of ROS 2 Humble.
`sudo apt install ros-dev-tools`	Installs development tools for ROS 2.
`sudo apt install ros-humble-turtlesim`	Installs the `turtlesim` package for ROS 2 Humble.
`sudo apt install python3-colcon-common-extensions`	Installs common extensions for `colcon` to facilitate package building.
`sudo apt install terminator`	Installs the Terminator terminal emulator.

Table A.I: Commands for system package management and updates in Ubuntu.

A.2 ROS 2 Environment Setup

Command	Description
`source /opt/ros/humble/setup.bash`	Sets up the environment for ROS 2 Humble in the current terminal session.
`source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash`	Sets up the autocompletion for `colcon` in the current terminal session.
`echo "source/opt/ros/humble/setup.bash" >> ~/.bashrc`	Adds the ROS 2 Humble environment setup to the `.bashrc` file for future terminal sessions.
`echo "source/usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash" >> ~/.bashrc`	Enables autocompletion for `colcon` in the `.bashrc` file for future terminal sessions.
`source ~/.bashrc`	Reloads the `.bashrc` file to apply changes without restarting the terminal.
`cat ~/.bashrc`	Displays the contents of the `.bashrc` file.
`gedit ~/.bashrc`	Opens the `.bashrc` file in the `gedit` text editor.

Table A.II: Commands for ROS 2 environment setup.

A.3 Workspace Creation and Building

Command	Description
`mkdir -p <ws_name>/src`	Creates the workspace directory and its `src` subdirectory.
`cd <ws_name>`	Navigates to the workspace directory.
`colcon build`	Builds all packages in the workspace.
`colcon build --symlink-install`	Builds packages using symbolic links, useful for development purposes.
`source ./install/setup.bash`	Sets up the environment for the built workspace in the current terminal session.
`code .`	Opens the current directory in Visual Studio Code.

Table A.III: Commands for workspace creation and building in ROS 2.

A.4 Package Creation and Editing in C++ and Python

Command	Description
`cd <ws_name>/src`	Navigates to the workspace's `src` directory.
`ros2 pkg create <pkg_name> --build-type ament_cmake --dependencies rclcpp std_msgs --license Apache-2.0 --description "Your package description here"`	Creates a new C++ package with specified dependencies, license, and description.
`ros2 pkg create <pkg_name> --build-type ament_python --dependencies rclpy std_msgs --license Apache-2.0 --description "Your package description here"`	Creates a new Python package with specified dependencies, license, and description.
`ls`	Lists the files and directories in the current directory.
`cd ..`	Navigates one directory up.
`rosdep install -i --from-path src --rosdistro humble -y`	In the root of the workspace, checks for missing dependencies before building.
`colcon build --packages-select <pkg_name>`	Builds only the `<pkg_name>` package.
`colcon build --packages-select <pkg_name> --symlink-install`	Builds the `<pkg_name>` package with symbolic link installation.
`cd src/<pkg_name>`	Navigates to the `<pkg_name>` directory.
`code package.xml`	Opens the `package.xml` file in Visual Studio Code.
`code CMakeLists.txt`	Opens the `CMakeLists.txt` file in Visual Studio Code.
`code setup.py`	Opens the `setup.py` file in Visual Studio Code.
`cd <ws_name>/src/<pkg_name>/src`	Navigates to the `src` subdirectory of a C++ package.
`cd <ws_name>/src/<pkg_name>/<pkg_name>`	Navigates to the `<pkg_name>` subdirectory of a Python package.
`touch <filename>.cpp`	Creates a new C++ source file named `<filename>`.
`touch <filename>.py`	Creates a new Python source file named `<filename>`.
`cd ../../..`	Navigates three directories up.

Table A.IV: Package creation and editing in C++ and Python.

A.5 Dev & Debug Essentials: CLI/GUI Commands

Command	Description
`ros2 run <pkg_name> <node_name>`	Runs the specified node from the specified package.
`ros2 run <pkg_name> <node_name> <args>`	Runs the specified node from the specified package with specified arguments.
`ros2 service call <service_name> <service_type> <args>`	Calls the service `<service_name>` from the specified package with specified arguments.
`ros2 node list`	Lists all active nodes in the ROS 2 system.
`ros2 node info <node_name>`	Displays detailed information about the specified node.
`ros2 topic list`	Lists all active topics in the ROS 2 system.
`ros2 topic info <topic_name>`	Displays detailed information about the specified topic.
`ros2 topic echo <topic_name>`	Echoes the messages of a specified topic.
`ros2 topic pub <topic_name> <msg_type> <args>`	Publishes a message to the specified topic.
`ros2 topic hz topic_name`	Displays the message frequency of a specified topic.
`ros2 interface show <pkg_name>/msg/<custom_msg_name>`	Shows the field structure of a custom message type in the specified package.
`ros2 interface show <pkg_name>/srv/<custom_srv_name>`	Shows the request/response structure of a custom service type in the specified package.
`ros2 run rqt_graph rqt_graph rqt_graph`	Runs the `rqt_graph` tool to visualize the ROS 2 nodes and topics.
`ros2 run rqt_service_caller rqt_service_caller`	Runs the `rqt_service_caller` tool to call services in a GUI.

Table A.V: Essential command-line and graphical interface commands for running nodes, interacting with topics and services, and debugging ROS 2 systems.

APPENDIX B

Using the `geometry_msgs` Package in C++ and Python

Author: Dr. Eduardo de Jesús Dávila Meza.
EduardoDavila-AI-PhD

This appendix provides an overview of how to utilize the `geometry_msgs` package in both C++ and Python. This package defines standardized message types for common geometric primitives such as points, vectors, quaternions, and poses, facilitating spatial data representation and interoperability throughout the ROS 2 ecosystem.

B.1 Importing the Package

B.1.1 In C++

To use the `geometry_msgs` in C++, include the necessary header files in your source code:

```
#include <geometry_msgs/msg/point.hpp>
#include <geometry_msgs/msg/quaternion.hpp>
#include <geometry_msgs/msg/pose.hpp>
```

Each message type corresponds to its own header file within the `geometry_msgs/msg` directory.

B.1.2 In Python

In Python, import the required message classes as follows:

```
from geometry_msgs.msg import Point, Quaternion, Pose
```

B.1.3 Including `geometry_msgs` in Your Package Dependencies

To use the `geometry_msgs` message types in your ROS 2 package, you must explicitly declare it as a dependency in both `CMakeLists.txt` (for C++) and `package.xml` (for both C++ and Python packages):

In `CMakeLists.txt`:

```
find_package(geometry_msgs REQUIRED)

ament_target_dependencies(<node_name> geometry_msgs)
```

In `package.xml`:

```
<depend>geometry_msgs</depend>
```

B.2 Using Message Definitions

The following examples demonstrate how to create, assign, and access values for the most commonly used message types from `geometry_msgs`.

B.2.1 Point

Represents a 3D position using Cartesian coordinates `x`, `y`, and `z`.

C++

```cpp
#include <geometry_msgs/msg/point.hpp>

geometry_msgs::msg::Point point;
point.x = 1.0;
point.y = 2.0;
point.z = 3.0;

// Accessing values
double x_value = point.x;
double y_value = point.y;
double z_value = point.z;
```

Python

```python
from geometry_msgs.msg import Point

point = Point()
point.x = 1.0
point.y = 2.0
point.z = 3.0

# Accessing values
x_value = point.x
y_value = point.y
z_value = point.z
```

B.2.2 Quaternion

Represents an orientation in free space using quaternion components `x`, `y`, `z`, and `w`. This representation avoids the gimbal lock issues found in Euler angles and supports smooth interpolations.

- `x`, `y`, and `z`: Vector part of the quaternion, defining the axis of rotation.
- `w`: Scalar part, representing the amount of rotation around the axis.
- A unit quaternion (with magnitude 1) is typically required for valid orientation.

C++

```cpp
#include <geometry_msgs/msg/quaternion.hpp>

geometry_msgs::msg::Quaternion quaternion;
quaternion.x = 0.0;
quaternion.y = 0.0;
quaternion.z = 0.0;
quaternion.w = 1.0;

// Accessing values
double w_value = quaternion.w;
```

Python

```python
from geometry_msgs.msg import Quaternion

quaternion = Quaternion()
quaternion.x = 0.0
quaternion.y = 0.0
quaternion.z = 0.0
quaternion.w = 1.0

# Accessing values
w_value = quaternion.w
```

B.2.3 Pose

Combines a `position` (as a Point) and `orientation` (as a Quaternion) to define a complete 6-DOF pose in 3D space.

C++

```cpp
#include <geometry_msgs/msg/pose.hpp>

geometry_msgs::msg::Pose pose;
pose.position.x = 1.0;
pose.position.y = 2.0;
pose.position.z = 3.0;
pose.orientation.x = 0.0;
pose.orientation.y = 0.0;
pose.orientation.z = 0.0;
pose.orientation.w = 1.0;

// Accessing values
double pos_x = pose.position.x;
double orient_w = pose.orientation.w;
```

Python

```python
from geometry_msgs.msg import Pose

pose = Pose()
pose.position.x = 1.0
pose.position.y = 2.0
pose.position.z = 3.0
pose.orientation.x = 0.0
pose.orientation.y = 0.0
pose.orientation.z = 0.0
pose.orientation.w = 1.0

# Accessing values
pos_x = pose.position.x
orient_w = pose.orientation.w
```

B.3 Additional Message Types

The `geometry_msgs` package includes other message types such as `Twist`, `Accel`, and `Wrench`, each with specific fields for representing different geometric concepts. The usage pattern for these messages is similar: import the message type, create an instance, and assign or access the relevant fields accordingly.

For more detailed information on each message type, refer to the ROS 2 official reference: geometry_msgs: Humble.

APPENDIX C

Configuring ROS 2 Package Paths for URDF Visualizer in VS Code

Author: Dr. Eduardo de Jesús Dávila Meza.
EduardoDavila-AI-PhD

To correctly visualize URDF or Xacro files that utilize `package://` URIs (Uniform Resource Identifiers), it is essential to configure the URDF Visualizer extension in Visual Studio Code to recognize the paths of your ROS 2 packages. This ensures that all resources referenced in your robot description files are accurately located and rendered.

C.1 Steps to Add a ROS 2 Package Path

1. Open your project folder in Visual Studio Code.
2. Navigate to the Extensions view by clicking on the Extensions icon in the Activity Bar or pressing `Ctrl+Shift+X`.
3. Search for URDF Visualizer and ensure it is installed.
4. Click on the gear icon (Manage) next to the URDF Visualizer extension and select **Extension Settings**.
5. In the settings, locate the **Urdf-visualizer: Packages** section.
6. Click on **Edit in settings.json** to open the `settings.json` file.
7. Within the `urdf-visualizer.packages` object, add your package path without removing existing entries. You can specify:
   - A relative path:
     ```
     "package_name": "${workspaceFolder}/src/package_name"
     ```
   - An absolute path:
     ```
     "package_name": "/absolute/path/to/package_name"
     ```

8. Save the `settings.json` file and close it.
9. To visualize your URDF or Xacro file:
    - Open the file in VS Code.
    - Click on the eye icon in the top-right corner.
    - Alternatively, press `Ctrl+Shift+P` to open the Command Palette, type `URDF:Preview URDF/Xacro`, and select it.
10. If the URDF Visualizer is already open, click on the reload button to apply the new settings.

C.2 Additional Notes

- If your URDF or Xacro file references multiple packages, ensure all are listed in the `urdf-visualizer.packages` configuration.
- Use absolute paths if your packages are located outside the current workspace.
- The extension supports both URDF and Xacro files, provided the package paths are correctly configured.

For more detailed information, refer to the official documentation: URDF Visualizer Extension.